

Decoupling & Reusability Plan

Decoupling & Reusability Plan

Scope: Define module boundaries, dependency inversion, shared libraries, platform abstraction layers, and how other projects can reuse each HelixVPN component. **Status:** Aligned with the MVP final spec and the current submodule layout.

1. Principles

1. **One core per concern, reused everywhere.** The Rust transport/VPN core is shared by client, connector, and edge. The Flutter UI core is shared by Access, Connector, and Console. Protobuf-generated clients are shared by Go/Dart/Rust.
 2. **Schema-first, zero drift.** The wire contract lives in helix-proto; generated stubs are the only clients. No hand-written parallel structs.
 3. **No hardcoded project identity.** Reusable crates contain no consumer project name, fixed hostname, tenant assumption, or asset path. Project specifics enter via configuration injection.
 4. **Dependency inversion.** Higher layers depend on trait interfaces, not on concrete transport or platform implementations.
 5. **Flat submodule layout.** Every reusable component is a direct child of submodules/; no nested own-org submodule chains.
-

2. Module boundaries

2.1 Rust core workspace (helix_core)

Crate	Boundary	Public interface	What it hides
helix-transport	L2 carrier abstraction	Transport, Connection, Listener, TransportRegistry, TransportError	UDP sockets, future TCP/TLS implementations
helix-wg	WireGuard crypto core	WireguardDevice, PeerConfig, WgConfig, key types, handshake helpers	boringtun::noise::Tunn, Noise state machine
helix-masque	MASQUE/QUIC carrier	MasqueTransport, MasqueConfig, MasqueConnection	quinn, TLS, HTTP-Datagram framing
helix-tun	OS TUN abstraction	TunDevice, TunConfig	Linux tun crate, platform differences
helix-orch	Tunnel lifecycle	Orchestrator, EventBus, TunnelEvent, TunnelState, wg_session	Three-loop scheduling, handshake driver
helix-core	Integration crate	Re-exports + map.json reconciler	Composition glue
helix-ffi (future)	Dart/UniFFI surface	helix_start, helix_stop, helix_status_stream	flutter_rust_bridge / UniFFI internals

Boundary rule: Crates inside helix_core may depend only on sibling crates at their abstraction level or below. helix-orch depends on helix-transport and helix-wg; helix-masque depends only on helix-transport and QUIC libraries.

2.2 Gateway edge (helix_edge)

Module	Boundary	Public interface	What it hides
edge	Edge listener binding	EdgeConfig, EdgeAddrs, spawn_edge()	MASQUE + decoy co-binding on same port
gateway	MASQUE-to-gateway relay	relay()	UDP socket forwarding,

Module	Boundary	Public interface	What it hides
decoy	TCP masquerade	bind(), run()	kernel-WG handoff point Static HTML response, probe handling

Boundary rule: `helix_edge` depends on `helix_core` crates via path dependencies and treats the gateway UDP address as an injectable endpoint. This lets the same relay logic talk to kernel WG, boringtun, or a test stand-in.

2.3 Go control plane (`helix_go`)

Domain service	Boundary	Public interface	What it hides
identity	Enrollment + OIDC	Coordinator.Enroll, Session.Authenticate	Token hashing, OIDC validation
registry	Devices + tenants	Device CRUD, cert serial lookup	Postgres RLS queries
ipam	Overlay addressing	AllocOverlayIP, ReleaseOverlayIP	ULA /48 + 4via6 allocation
pki	Certificates	IssueDeviceCert, RevokeCert	CA, short-lived mTLS
policy	ACL compiler	CompilePolicy, VisibleTo	Tailscale-ACL-flavored rules
coordinator	Map streaming	Coordinator.WatchNetworkMap	In-memory graph, delta computation
events	Event bus	Publish/subscribe on Redis Streams	Redis topology, consumer groups
telemetry	Metrics/logs	Telemetry.SubmitMetrics, SubmitLog	sampling, anonymization

Boundary rule: Services communicate through domain events and the Coordinator proto, not by direct imports except inside the API gateway layer.

2.4 Protobuf (helix_proto)

Package	Scope	Consumers
helix.coordinator.v1	Agent $\rightarrow$ control-plane contract	Go server, Rust/Dart agents
helix.session.v1	Console auth + session validation	Go API gateway, Console REST
helix.tunnel.v1	Core $\rightarrow$ UI status/events	Rust FFI, Dart UI, telemetry collector
helix.ui.v1	Normalized UI state	Flutter widgets, Challenge/QA harness
helix.telemetry.v1	Aggregate metrics/logs	Edge, agents, telemetry collector

3. Dependency inversion

3.1 Transport trait

The orchestrator and WG layer depend on the abstract Transport trait, not on UdpTransport or MasqueTransport. New carriers are added by implementing the trait and registering them in the ladder policy.

Current code has a different trait shape; the plan is to converge on the frozen spec trait:

```
pub trait Transport: Send + Sync {
    async fn send(&self, datagram: Bytes) -> Result<(),
        TransportError>;
    async fn recv(&self) -> Result<Bytes, TransportError>;
    fn kind(&self) -> &'static str;
    fn effective_mtu(&self) -> u16;
    fn health(&self) -> TransportHealth;
    async fn close(&self) -> Result<(), TransportError>;
}

pub async fn dial(cfg: TransportConfig) -> Result<Box<dyn
    Transport>, TransportError>;
```

The dial() free function selects the concrete implementation from the TransportConfig enum, keeping the orchestrator agnostic of transport internals.

3.2 Platform tunnel shim

The Rust core treats the TUN as an injectable PacketIO trait:

```
#[async_trait]
pub trait PacketIO: Send + Sync {
    async fn read_packet(&self, buf: &mut [u8]) -> Result<usize>;
    async fn write_packet(&self, buf: &[u8]) -> Result<usize>;
    fn mtu(&self) -> u16;
}
```

helix-tun provides the Linux implementation. Platform shims (helix_shims) provide Android VpnService/JNI, Apple NEPacketTunnelProvider, Windows wireguard-nt, HarmonyOS Network Kit, and Aurora Qt/tun implementations, all satisfying the same trait.

3.3 PKI abstraction

The control plane depends on a PkiService trait, not on a specific CA library:

```
type PkiService interface {
    IssueDeviceCert(ctx context.Context, deviceID uuid.UUID, ttl
        time.Duration) ([]byte, error)
    Revoke(ctx context.Context, serial string) error
}
```

This allows reuse in projects with different CA backends (internal CA, ACME, HashiCorp Vault).

4. Shared libraries

Library	Location	How to reuse
Pluggable transport abstraction	helix_core/crates/ helix-transport	Add crate dependency; implement Transport for new carriers
WireGuard wrapper	helix_core/crates/ helix-wg	Use key types, config, handshake helpers
QUIC + MASQUE framing	helix_core/crates/ helix-masque	Use quinn endpoints and HTTP-Datagram codec
Linux TUN	helix_core/crates/ helix-tun	Use TunDevice for any Linux VPN product
Event bus vocabulary		

Library	Location	How to reuse
	helix_core/crates/ helix-orch + helix.tunnel.v1 proto	Consume TunnelEvent in any UI/telemetry consumer
RFC 9298 edge terminator	helix_go/pkg/ masqueedge	Import package; inject listen address, TLS, and gateway target
Agent control contract	helix_proto	Generate stubs for Go/ Dart/Rust/TypeScript

5. Platform abstraction layers

Platform	Shim responsibility	Trait boundary
Linux	Create/manage TUN interface, set routes	PacketIO
Android	VpnService + JNI fd handoff	PacketIO
iOS / macOS	NEPacketTunnelProvider + NetworkExtension	PacketIO + UI status channel
Windows	wireguard-nt + privileged service	PacketIO
HarmonyOS NEXT	Network Kit ability	PacketIO
Aurora OS	Qt/C++ + tun fd	PacketIO
Web (WASM)	Browser-scoped MASQUE proxy, NOT system VPN	WebTransport fallback

The Rust core stays platform-agnostic. Only the shim layer changes per platform.

6. How other projects can reuse each component

6.1 Reusing helix_core

A third-party VPN or SD-WAN product can depend on helix_core crates to obtain a Rust-based WG + pluggable transport stack without adopting the HelixVPN control plane. Required inputs:

- WgConfig + peer public keys
- TransportConfig (plain-udp, masque-h3, etc.)
- A PacketIO implementation for the target platform

Caveat: the current Transport trait shape is not yet final. External projects should wait for the spec reconciliation or pin a specific commit.

6.2 Reusing helix_edge

A self-hosted gateway project can use `helix_edge::edge::spawn_edge()` to get a MASQUE-terminating edge with a TCP decoy on the same port. Required inputs:

- Bind address + SNI host
- TLS certificate chain + private key
- Gateway UDP target (kernel WG / boringtun / test socket)

Caveat: the Rust MASQUE implementation currently uses a labeled stand-in for CONNECT-UDP flow establishment. For a turnkey RFC 9298 solution, prefer `helix_go/pkg/masqueedge` today.

6.3 Reusing helix_go/pkg/masqueedge

Any Go project needing a CONNECT-UDP proxy/terminator can import `github.com/vasic-digital/helix_go/pkg/masqueedge`. It requires only:

- Listen address
- `*tls.Config`
- Gateway UDP target passed via the URI template

6.4 Reusing helix_proto

Any project needing the agent $\rightleftharpoons$ control-plane contract can copy the `proto/helix/...` tree and run `buf generate` with its own language plugins. The package names and field numbers are stable for v1.

7. Current coupling risks and mitigation

Risk	Severity	Mitigation
Transport trait shape mismatch	High	Reconcile to frozen spec before external adoption; add migration guide
<code>helix_transport</code> scaffolding duplicates	Medium	Decide: archive, merge, or make it a re-export; document decision

Risk	Severity	Mitigation
helix_core/crates/ helix-transport		
helix_edge hard path-depends on helix_core layout	Low	Already tested by dependency_resolution_smoke.rs; keep sibling layout stable
Go control plane does not exist yet	High	Implement as modular monolith with interface boundaries (§3.3)
Dart/Rust proto stubs not generated	Medium	Install plugins and add buf generate to pre-build hook
No platform shims beyond Linux	High	Implement behind PacketIO trait per platform

8. Recommended next steps

1. Converge Transport trait to the frozen spec and migrate all current implementations/tests.
2. Generate Dart + Rust proto stubs and replace any hand-written structs.
3. Implement the Go control-plane services behind the trait boundaries above.
4. Add PacketIO trait and migrate helix-tun + future shims behind it.
5. Resolve the helix_transport submodule disposition.
6. Add buf breaking CI gate against main to protect the v1 field-number registry.

9. Links

- System architecture overview: README.md (sibling file)
- API contracts: docs/research/mvp/final/implementation/08-api-contracts/README.md
- Alignment report: docs/reviews/mvp-final/findings/phase3-code-spec-alignment.md
- Protobuf source: submodules/helix_proto/proto/helix/...